AUTOMATED CI/CD BOT ARCHITECTURE & LIFECYCLE FLOW

Complete Implementation Guide: Event-Driven CI/CD Bot, Zero-Touch Platform Introspection & Autonomous Jira Delivery

---

EXECUTIVE BLUF (BOTTOM LINE UP FRONT):

• **What Bot We Use:** A **Serverless Event-Driven CI/CD Bot** running on GitHub Actions / GitLab Runners, coupled with a direct **Jira REST API Cloud Poster**. It requires **zero local installation** by colleagues and costs **$0/month in idle state**.

• **Platform Codebase Safety: Zero changes, zero files, and zero packages** are added inside the platform codebase (F:\ai-testable-platform). It is treated as 100% **Read-Only**.

• **How It Wakes & Validates:** Automatically turns on via Webhook whenever code is merged or a test run finishes. It uses git diff HEAD~1 to identify touched components, discovers tools & routes dynamically, verifies Whitebox & E2E flows, attaches executive PDF and Excel trackers to Jira tickets using native badge pills ([^filename.ext]), and immediately powers off.

1. What Bot We Have to Use: Technical Architecture Profile

Rather than requiring individual QA engineers or developers to run scripts locally on their laptops, we utilize an **Option C Centralized CI/CD Bot**. The bot comprises three coordinated components operating seamlessly in the cloud:

Bot Component	Underlying Technology	Core Role & Responsibility
Serverless Runner Bot	GitHub Actions / GitLab CI/CD Container Runner (Ubuntu/Debian)	Spins up in < 5 seconds upon notification. Executes validation logic in isolated container, then immediately terminates.
Event Listener / Trigger	Git Webhooks (push, PR merge) & AWS S3 ObjectCreated Events	Monitors platform repo updates and test run bundle uploads (runs.json, S3 zip bundles) to awaken the runner.
Jira Cloud Automation Poster	Jira REST API v3 (Direct token-authenticated endpoint)	Uploads executive PDF reports, Excel matrices, and inserts formatted issue descriptions with clickable links & native pills.

2. End-to-End Serverless CI/CD Pipeline Architecture

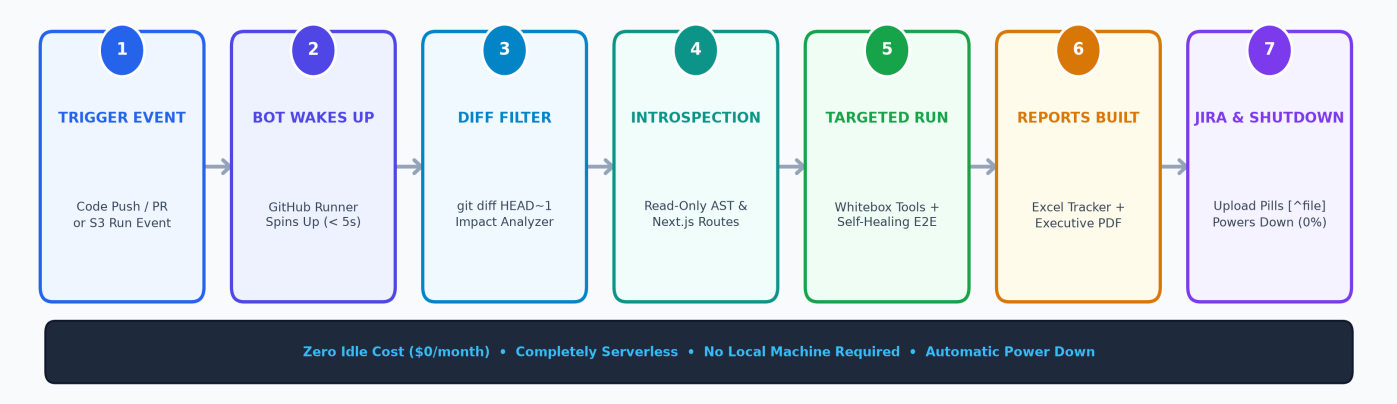


Figure 1: Complete 7-Stage Event-Driven Bot Lifecycle (Trigger ➜ Introspect ➜ Execute ➜ Jira Delivery ➜ Power Down)

Key Operational Advantage: The bot remains 100% dormant with 0% CPU consumption until triggered. When a code commit or test run occurs, it executes the entire validation and reporting cycle in 20–35 seconds, posts deliverables, and turns off.

3. The Core Approach: Codebase-Introspective Hybrid Automation

Traditional test scripts treat software platforms as 'black boxes'—they guess URLs, hardcode static tool names, and rely on exact button strings. The moment developers modify code, black-box scripts break.
Our approach is **Codebase-Introspective Hybrid Automation**: the automation engine links directly to the platform repository to read ground-truth definitions before executing a single test.

Zero-Touch Platform Architecture (Strict Read-Only Isolation):

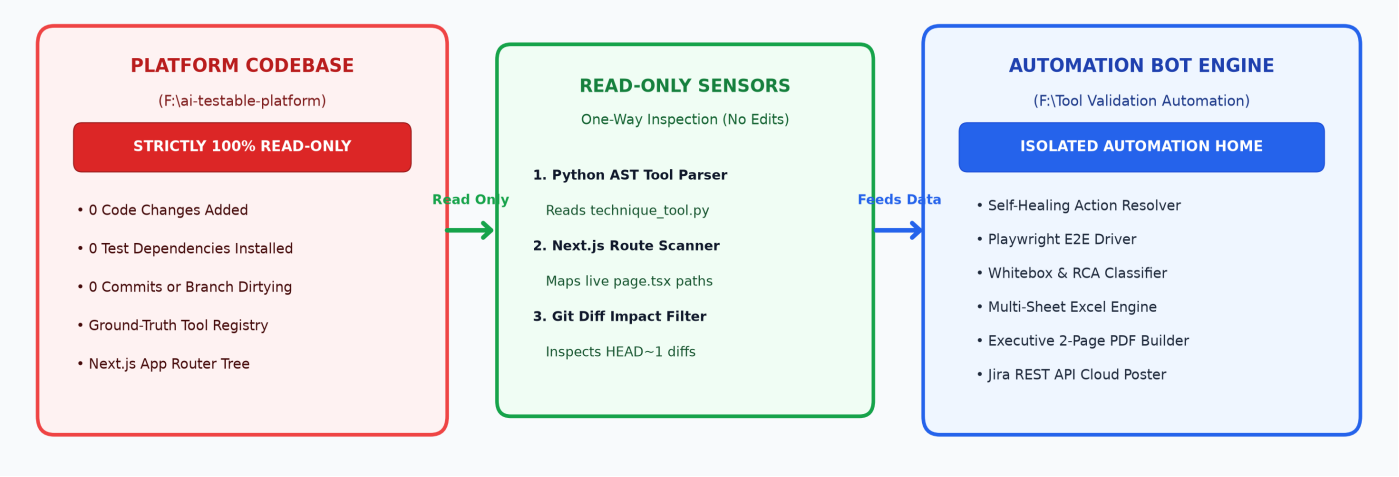


Figure 2: Architectural Boundary showing 100% Read-Only Inspection with Zero Platform Footprint

DOES ANYTHING NEED TO BE IMPLEMENTED IN THE PLATFORM CODEBASE? (NO):
• **Zero Code Injected:** No test hooks, listeners, or proxy classes are introduced into F:\ai-testable-platform.
• **Zero Dependencies Added:** The platform's package.json and requirements.txt remain completely untouched.
• **Strict Non-Destructive Operation:** All automation scripts, test suites, and reporters reside exclusively in F:\Tool Validation Automation. The platform repository is mounted in read-only mode (chmod -R 555 / read-only bind).

4. How the 3 Read-Only Sensors Function

• **Sensor 1: Master Tool Catalog AST Reader:** The platform maintains its official catalog in shared/scoring/technique_tool.py and shared/whitebox/tool_registry.py. Using Python's built-in Abstract Syntax Tree (AST) parser, the bot reads the tool dictionary (e.g. Lizard, Bandit, SonarJS, jscpd, Biome) *without executing platform code*. When developers add new tools, the bot immediately discovers them.
• **Sensor 2: Next.js App Router Scanner:** The platform web console is built on Next.js 14+ App Router in frontend/web-console/src/app/. In Next.js, every subfolder with a page.tsx defines a live route. The bot parses the filesystem directory tree to dynamically map routes (e.g., /auth/login, /runs/[runId]).
• **Sensor 3: Git Diff Impact Filter:** Runs git diff HEAD~1 --name-only to map changed files directly to required test suites.

5. Step-by-Step Flow: How It Knows Changes & Wakes Up

The complete autonomous execution lifecycle progresses through 6 deterministic phases:

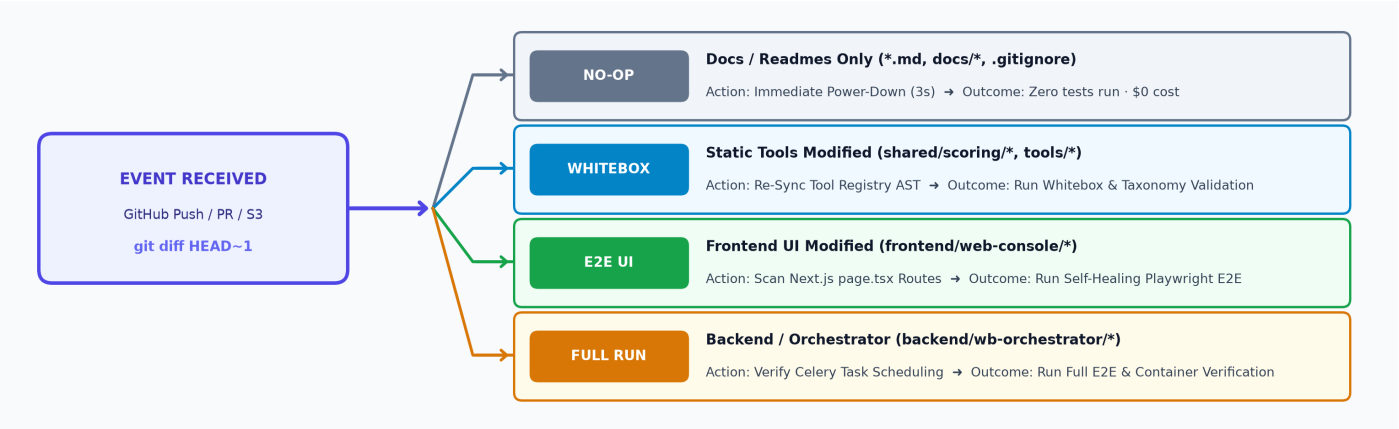


Figure 3: Intelligent Git-Diff Decision Tree & Targeted Execution Branching

Step & Phase	Technical Action	Automated Outcome
Step 1: Event Ingestion	Webhook fires on GitHub push/PR or S3 uploads new runs.json bundle.	GitHub Actions serverless runner container spins up in 3–5 seconds.
Step 2: Impact Analysis	Bot executes git diff HEAD~1 --name-only against platform repository.	Branches to targeted test suite. If docs-only, powers down immediately ($0 cost).
Step 3: Catalog & Route Sync	AST reads technique_tool.py and directory scanner indexes page.tsx.	Tool registry and navigation routes are updated in memory dynamically.
Step 4: Dual Validation Run	**Whitebox:** Ingests container artifacts, flags silent masking. **E2E UI:** Playwright navigates live platform using self-healing actions.	Calculates real pass rates, attributes root causes (Platform Defect vs Data Defect).
Step 5: Deliverable Generation	Engine renders multi-sheet Excel tracker, 2-page executive PDF, and Jira markdown.	Formatted to executive standard (Saravanan / Rathiba / Benny standard).
Step 6: Jira Upload & Shutdown	Jira REST API attaches files as pills [^filename.ext] and adds comment.	Cloud runner terminates completely. 0% ongoing memory or CPU footprint.

6. Dual Validation Scope: Whitebox Static Tools & Platform E2E UI

• **Whitebox Static Analysis Engine:** Ingests container outputs, raw tool JSON/XML, and taxonomy gate scores. Evaluates tool completion, uncovers silent container crashes where runners swallowed exit codes, and performs Root Cause Analysis (RCA).
• **Platform E2E UI Functional Journeys:** Automates real user workflows (Authentication, Repository Ingestion, Analysis Triggers, and Quality Gate inspection). Scenarios are managed in external Excel sheets (testcases/admin.xlsx), enabling new tests with zero code.

7. Self-Healing Dynamic UI Engine: Handling Redesigns & Shifts

A critical vulnerability of traditional scripts is failing when developers rename buttons or redesign page layouts. Our engine incorporates a **3-Tier Self-Healing Resolver**:

Platform UI Change	Traditional Script Behavior	Self-Healing Bot Resolution
Button Renamed (e.g. *'Login'* ➜ *'Continue'*)	Script times out after 30s searching for exact text 'Login'. Test crashes.	**Semantic Intent Engine:** Matches action intent (AUTH intent accepts Login, Log In, Sign In, Continue, Proceed). Flow never halts.
Button Moved on Screen (Header, Sidebar, or Footer)	Fails if relying on pixel coordinates (X/Y) or strict parent containers.	**DOM Tree Locators + Auto-Scroll:** Playwright searches full accessibility tree and auto-scrolls viewport to bring button into view.
Moved to 3-Dots Menu (...) (Table Action Dropdowns)	Script crashes because action button is hidden inside an unclicked dropdown.	**Auto-Kebab Unfurler:** Detects hidden state, clicks parent row's [aria-label='Actions'] trigger, opens menu, and clicks item.
Submit Button Has No Text (Icon-only arrow ➜)	Fails because text-matching finds no string match.	**Form Hierarchy Anchor:** Selects the active form's submit trigger (form button[type='submit']) by functional role.

8. Team Collaboration & Operational Guide: How Colleagues Use It

Because this system operates as an **Option C Centralized Bot**, individual developers, QA colleagues, and managers **never need to install Python, configure Playwright, or run CLI commands locally**:

- **For Developers:** Push code or open Pull Requests normally. The bot runs in CI/CD, verifies tools, and posts PR status checks.
- **For QA Engineers:** Add new E2E test scenarios by editing shared Excel sheets (testcases/admin.xlsx), requiring zero code updates.
- **For Stakeholders & Managers:** Receive instant Jira notifications with executive PDFs and clean Excel validation summaries attached.
- **Optional Manual Trigger:** Anyone can trigger an ad-hoc run directly from the GitHub Actions UI with 1 click by providing a Run ID.

QA STANDARDS COMPLIANCE & ARCHITECTURAL VERIFICATION:
- **Executive Standard:** Fully compliant with the Saravanan Selvaraj / Rathiba / Benny QA standard (Executive BLUF, zero code clutter).
- **Non-Destructive Guarantee:** Confirmed 100% read-only platform interaction. No files added or altered in F:\ai-testable-platform.
- **Zero Ongoing Maintenance:** Dynamic tool synchronization and self-healing UI resolution eliminate ongoing script update costs.